



Année universitaire 2025/2026

NOM :

PRENOM :

Consignes relatives au déroulement de l'épreuve

Date : 21 avril 2026

Devoir Module Mise en œuvre d'un système à microprocesseur – Session 1 - **Partie TP**

Durée: 2h30

Professeurs responsables : Nacer Abouchi - François Joly

Documents : ☒ autorisés ☐ non autorisés

Si oui : type(s) de documents autorisés : Polycopié Fiche technique du 8051F020 uniquement

Calculatrices : ☒ autorisées ☐ non autorisées

Si oui : type(s) de calculatrices autorisées : alphanumériques

LES TELEPHONES PORTABLES ET AUTRES APPAREILS DE STOCKAGE DE DONNEES NUMERIQUES NE SONT PAS AUTORISES.

Les téléphones portables doivent être éteints pendant toute la durée de l'épreuve et rangés dans les cartables.

S'agissant de contrôle sans document, les trousseaux doivent être rangés dans les cartables.

Les cartables doivent être fermés et posés au sol.

Les oreilles des candidats doivent être dégagées.

Rappels importants sur la discipline lors des examens

La présence à tous les examens est strictement obligatoire ; tout élève présent à une épreuve doit rendre une copie, même blanche, portant son nom, son prénom et la nature de l'épreuve.

Une absence non justifiée à un examen invalide automatiquement le module concerné.

Toute suspicion sur la régularité et le caractère équitable d'une épreuve est signalée à la direction des études qui pourra décider l'annulation de l'épreuve; tous les élèves concernés par l'épreuve sont alors convoqués à une épreuve de remplacement à une date fixée par le responsable d'année.

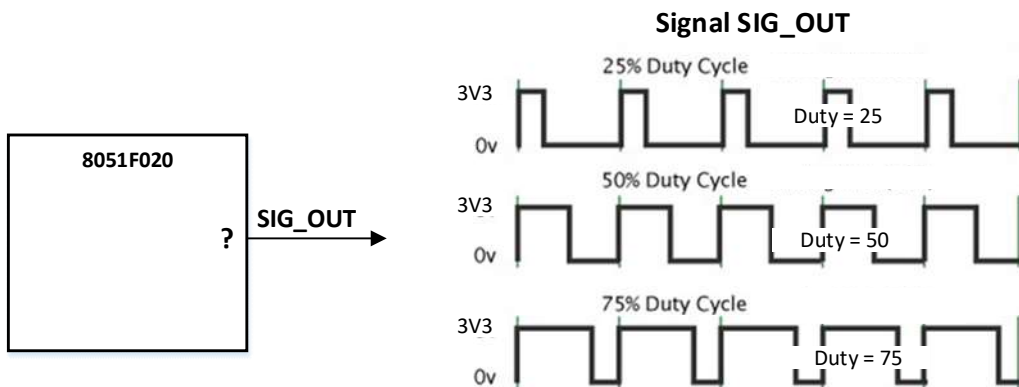
Toute fraude ou tentative de fraude est portée à la connaissance de la direction des études qui pourra réunir le Conseil de Discipline. Les sanctions prises peuvent aller jusqu'à l'exclusion définitive du (des) élève(s) mis en cause.

Devoir Session 1 - **Partie TP**

Durée de l'épreuve : 2,5 heures

Analyse de code (14 points)

Vous trouverez en annexe un code en langage C qui permet de produire un signal SIG_OUT qui produit une modulation de largeur d'impulsion à la fréquence de 5KHz. Ainsi, le signal SIG_OUT est un signal numérique 0/3,3V dont on peut faire varier le rapport cyclique (Duty) c'est-à-dire, le rapport entre la durée du niveau haut et la durée du niveau bas. Nous vous demandons d'analyser ce code dans un premier temps, puis vous devrez y apporter des modifications.



1. Expliquez la signification de la ligne 19 « `sfr16 T2 = 0x0c;` » (0,5 point)

2. Expliquez la signification de la ligne 20 « `sbit Flag_ISR = P2^5;` » (0,5 point)

3. Expliquez la signification de la ligne 11 « `#define SYSCLK 23961600L` » Produit-elle du code ? (0,5 point)

4. Sur quelle broche du microcontrôleur pourra-t-on observer le signal de sortie en modulation de largeur d'impulsion SIG_OUT? Comment est configurée la broche ? Et pourquoi ? (0,5 point)

5. Pourquoi dévalide-t-on les interruptions durant l'exécution des lignes de code 136 et 137 dans la fonction Main – Expliquez – Justifiez. (0,5 point)

6. Analyse de la fonction `Timer_Init` – Quel est le Timer mis en œuvre dans cette fonction? Précisez sa configuration – Peut-il produire des interruptions ? Si oui, quel est l'évènement qui pourra produire des interruptions ? (2 points)

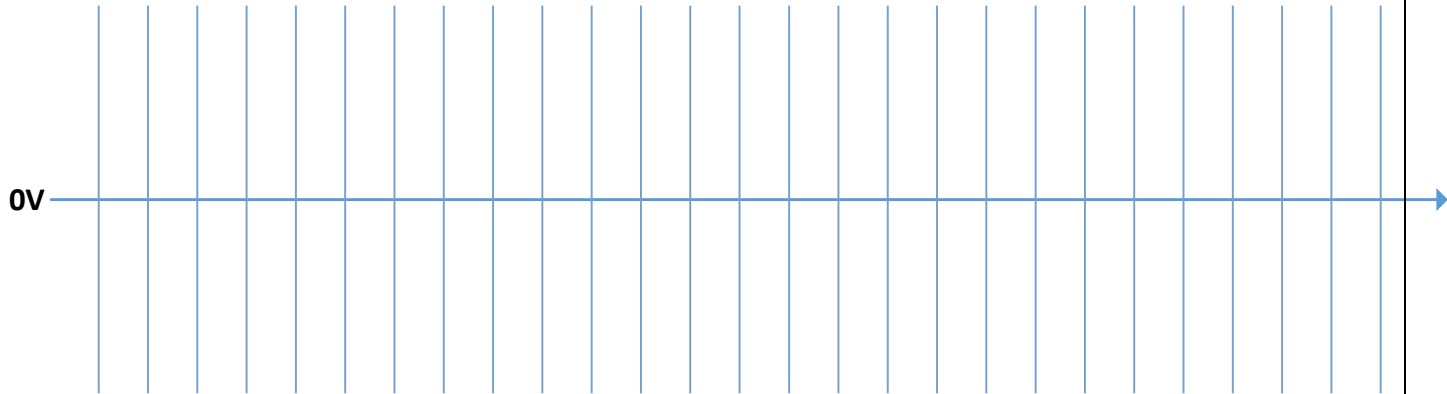
10. Ce code a été conçu pour une horloge système à 23,9616MHz. Indiquer toutes les modifications à apporter pour fonctionner à 22,1184MHz. (1 point)

11. Expliquez le rôle des 3 lignes de code : « XBR0 = 0x04; XBR1 = 0x94; XBR2 = 0x44; » dans la fonction Port_IO_Init. (1,5 point)

12. La variable « Duty » est en fait mise à jour depuis l'extérieur du microcontrôleur. Comment ? Expliquez. (1 point)

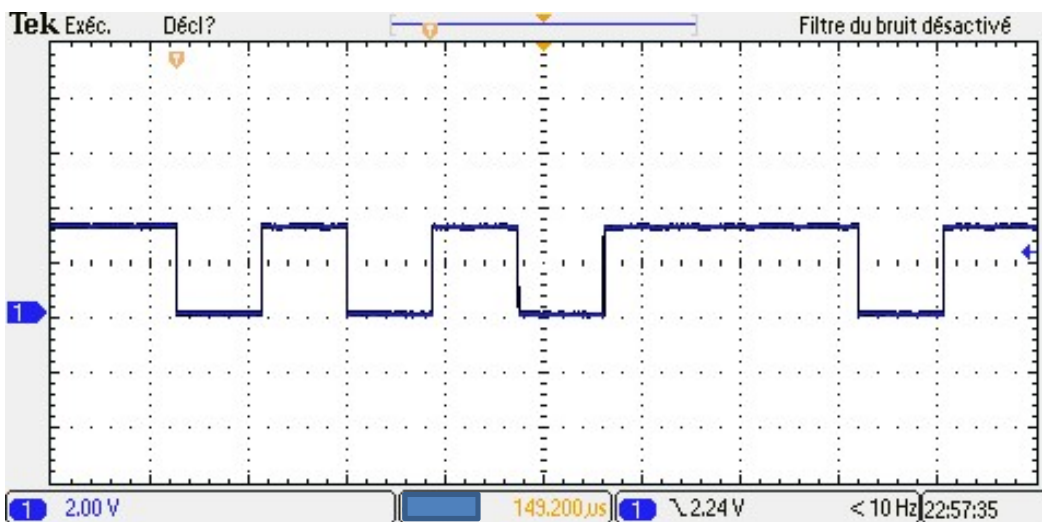
16. **Transmission série – 1** (1 point)

Dessiner sur le schéma ci-dessous la transmission sur une liaison UART en mode CMOS/TTL des 2 caractères ASCII : « 26 » de code ASCII 0x32 et 0x36. Faites apparaître sur le schéma ci-dessous que la transmission est à 1000 Baud. **Commentez votre tracé**



17. **Transmission série – 2** (1 point)

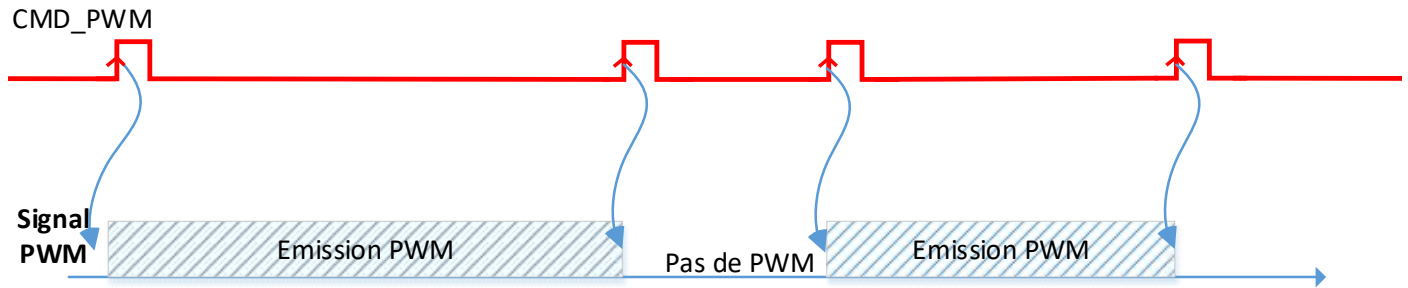
Lors de premier essais, après avoir branché la liaison série du PC sur le microcontrôleur, un premier octet est transmis par le PC. Cet octet transmis a été capturé par l'oscilloscope. Compte tenu de l'oscillogramme ci-dessous, déterminez la vitesse de transmission et la valeur de l'octet transmis. Remarque : la base de temps de l'oscilloscope est réglée sur 100µs/div



Amélioration du Code. (3 points)

On souhaite pouvoir commander la sortie en modulation de largeur d'impulsion à l'aide d'un signal impulsionnel 0/3,3V nommé CDE_PWM. A chaque fois que l'on aura une transition positive sur ce signal (front montant) on commutera l'état de la sortie du signal PWM (soit la sortie émet un signal PWM, soit elle n'émet rien). Proposez une solution pour résoudre ce problème.

Pas besoin d'écrire du code, mais expliquez plutôt ce que vous allez devoir mettre en place et ce que vous allez devoir modifier.




```

1 //*****
2 // Devoir 3IRC Achi 8051 - Session 1 - Avril 2026 -- Code_PWM.C
3 // Génération d'un signal SIG_OUT en modulation de largeur d'impulsion.
4 // On souhaite produire un signal à une fréquence de 5 Khz.
5 // Le rapport cyclique est fixé par une variable nommée "Duty" dont la valeur
6 // peut varier de 1 à 99
7 // "Duty" exprime en pourcentage la durée du niveau haut du SIG_OUT
8 //*****
9 #include "C8051F020.h"
10
11 #define SYSCLK 23961600L
12 #define CLK_Timer2 SYSCLK
13 #define LOW 0
14 #define HIGH 1
15 #define OFF 0
16 #define ON 1
17
18 sfr16 RCAP2 = 0xca;
19 sfr16 T2 = 0xcc;
20 sbit Flag_ISR = P2^5;
21 sbit SIG_OUT = P2^3;
22 unsigned int Preload_Pulse_H;
23 unsigned int Preload_Pulse_L;
24 unsigned int Pulse_H;
25 unsigned int Pulse_L;
26 char Duty = 50;
27 bit flag_new_Duty = ON;
28 //*****
29 void Reset_Sources_Init()
30 {
31     WDTCN = 0xFF;
32     WDTCN = 0x07;
33 }
34 //*****
35 void Port_IO_Init()
36 {
37     P0MDOUT = 0x45;
38     P2MDOUT = 0xFF;
39     P2 = 0;
40     P3MDOUT = 0x00;
41     P3 = 0xFF;
42     XBR0 = 0x04;
43     XBR1 = 0x94;
44     XBR2 = 0x44;
45 }
46 //*****
47 //*****
48 void Timer_Init()
49 {
50     T2CON = 0x04;
51     CKCON |= (1<<5);
52     T2 = Preload_Pulse_H;
53     RCAP2 = Preload_Pulse_H;
54     IE |= 0x20;
55     IP &= ~0x20;
56 }
57 //*****
58 //*****
59 void Serial_Init()
60 {
61     CKCON &= ~0x08;
62     TMOD |= 0x02;
63     TMOD &= ~0x0D;
64     TCON |= 0x40;
65     TH0 = 256-104;
66     T4CON &= ~0x60;
67     PCON |= 0x10;
68     PCON &= ~0x80;
69     SCON1 = 0x70;

```

```

70     EIE2      |= 0x40;
71     EIP2      &= ~0x40;
72 }
73 //*****
74 //*****
75 void Oscillator_Init()
76 {
77     int i = 0;
78     OSCXCN     = 0x67;
79     for (i = 0; i < 3000; i++);
80     while ((OSCXCN & 0x80) == 0);
81     OSCICN     = 0x0C;
82 }
83 //*****

84 void Init_Device(void)
85 {
86     Reset_Sources_Init();
87     Oscillator_Init();
88     Port_IO_Init();
89 }
90 //*****
91 unsigned int  Compute_Pulse_H(char Duty)
92 {
93     unsigned int value;
94     if (Duty < 1) Duty = 1;
95     else if (Duty > 99) Duty = 99;
96     value = ((CLK_Timer2/5000)*Duty)/100;
97     return value;
98 }
99 //*****
100 unsigned int  Compute_Pulse_L(char Duty)
101 {
102     unsigned int value;
103     if (Duty < 1) Duty = 1;
104     else if (Duty > 99) Duty = 99;
105     value = ((CLK_Timer2/5000)*(100-Duty))/100;
106     return value;
107 }
108 //*****

109 void Enable_Interrupt(void)
110 { EA=1; }
111 //*****

112 void Disable_Interrupt(void)
113 { EA=0; }
114 //*****

115 //Main
116 //*****

117 main()
118 {
119     Init_Device();
120     Timer_Init();
121     Serial_Init();
122     Enable_Interrupt();
123     Pulse_H = Compute_Pulse_H(Duty);
124     Pulse_L = Compute_Pulse_L(Duty);
125
126     while(1)
127     {
128         WDTCN     = 0xA5;
129         if (flag_new_Duty == ON)
130         {
131             flag_new_Duty = OFF;
132             Pulse_H = Compute_Pulse_H(Duty);
133             Pulse_L = Compute_Pulse_L(Duty);

```

```
134
135     Disable_Interrupt();
136     Preload_Pulse_H = 65536 - Pulse_H;
137     Preload_Pulse_L = 65536 - Pulse_L;
138     Enable_Interrupt();
139 }
140 }
141 }
142 //*****

143 void ISR_PWM (void) interrupt 5
144 {
145     Flag_ISR = 1;
146     if (TF2 == 1)
147     {
148         TF2 = 0;
149         if (SIG_OUT == LOW)
150         {
151             SIG_OUT = HIGH;
152             RCAP2 = Preload_Pulse_L;
153         }
154         else
155         {
156             SIG_OUT = LOW;
157             RCAP2 = Preload_Pulse_H;
158         }
159     }
160     if (EXF2 == 1) EXF2 = 0;
161     Flag_ISR = 0;
162 //*****

163 void ISR_Msg_Reception (void) interrupt 20
164 {
165     if ((SCON1 & 0x02) != 0) SCON1 &= ~0x02;
166
167     if ((SCON1 & 0x01) != 0)
168     {
169         SCON1 &= ~0x01;
170         Duty = SBUF1;
171         flag_new_Duty == ON;
172     }
173 }
```